

Name: _____

Answer all questions below. Show work for partial credit.

1. Assume one or more producer threads add `loops` different values into a buffer, and that one or more consumer threads remove `loops` values from the buffer.

(a) Assume the producer/consumer code is implemented using conditional waits. Assume the producer is implemented as the following :

```
for (i = 0 to loops) {  
    pthread_mutex_lock(&mutex);  
    while(buffer_size > limit)  
        pthread_cond_wait(&cond, &mutex);  
    put(values[i]);  
    pthread_mutex_unlock(&mutex);  
}
```

Also, assume the consumer is implemented as follows :

```
for (i = 0 to loops) {  
    pthread_mutex_lock(&mutex);  
    while(buffer_size == 0)  
        pthread_cond_wait(&cond, &mutex);  
    values[i] = get();  
    pthread_cond_signal(&cond);  
    pthread_mutex_unlock(&mutex);  
}
```

- i. What could go wrong with the code above?

- ii. How would you change the code to prevent this from happening?

- (b) Assume the producer/consumer code is implemented using semaphores. Assume the producer is implemented as the following :

```
for (i = 0 to loops) {  
    sem_wait(&empty);  
    put(values[i]);  
    sem_post(&full);  
}
```

Assume the consumer is implemented as the following :

```
for (i = 0 to loops) {  
    sem_wait(&full);  
    values[i] = get();  
    sem_post(&empty);  
}
```

- i. What could go wrong with the code above?
- ii. How would you change the code to prevent this from happening?

2. In what situation should you use `pthread_cond_broadcast` instead of `pthread_cond_signal`?

3. Suppose two threads want to allocate data, but there is not sufficient space remaining. A third thread already holds an allocation and will free that data. The threads are implemented as follows :

Thread 0 :

```
pthread_mutex_lock(&mutex);
while (10 > bytes_remaining)
    pthread_cond_wait(&cond, &mutex);
var = malloc(10);
bytes_remaining -= 10;
pthread_mutex_unlock(&mutex);
```

Thread 1:

```
pthread_mutex_lock(&mutex);
while (75 > bytes_remaining)
    pthread_cond_wait(&cond, &mutex);
var = malloc(75);
bytes_remaining -= 75;
pthread_mutex_unlock(&mutex);
```

Thread 2 :

```
pthread_mutex_lock(&mutex);
free(50);
bytes_remaining += 50;
pthread_cond_signal(&cond);
pthread_mutex_unlock(&mutex);
```

- (a) What could go wrong with the code above?

- (b) How would you change the code to prevent this from happening?

4. Suppose I have two types of threads. Type A writes to a global buffer while type B reads from a global buffer. Explain a situation where a race condition could occur.

5. Assume there are p threads, and also p locks. Each thread needs multiple locks before it can execute a critical section. It then can release the locks for other threads to use.

(a) What are the requirements that must be met for a deadlock to occur?

(b) Explain how you could break one of these requirements to prevent a deadlock.

6. Assume I have an openmp program and notice the time required to do the following :

```
#pragma omp atomic  
x += 10;
```

is always significantly cheaper than the following :

```
#pragma omp critical  
{  
    x += 10;  
}
```

Explain why this happens.

7. Assume I have 5 philosophers and 5 forks. Assume each philosopher needs both the fork to their left and the fork to their right before they are able to eat. Assume all philosophers want to eat. Explain a method that would guarantee all philosophers are able to eat without a deadlock.
8. Assume the inode size of my file system is 512 bytes. Also assume that the inodes are contiguous and start at 15KB. What is the location of inode 56 (in bytes). Hint $1\text{KB} = 1024$ bytes.
9. Assume my file system has a block size of 4KB (4096 bytes) and inode size of 512 bytes. In which iblock is inode 36 located?
10. Assume my file system has the following parameters :
- inode size : 512 bytes
 - block size : 4KB (4096 bytes)
 - sector size : 512 bytes
 - inode start address 15KB (15360 bytes)
- In which sector is inode 34 located?
11. Assume I want to open a **read-only** file `'/usr/local/src/tmp.txt'`. How many reads and writes (to disk) are required to open this file?

12. Assume I want to **create** and open a **write-only** file `'/usr/local/src/tmp.txt'`. How many reads and writes (to disk) are required to open this file?
13. What is the purpose of journaling?
14. What are the benefits of snapshots?